AMAR TELIDJI UNIVERSITY

Laghouat

Computer Sciences Department

Class: **4th Year Cybersecurity**

Teacher: **Dr. Nasreddine CHAIB**

Student: **Smail Mersad**

COURSE

Identity and Access Management

REPORT TITLE

Lab Sessions

2025 / 2026

1. Lab Objective

The objective of this lab is to understand how web sessions are created and managed using cookies, demonstrate a session fixation attack in insecure mode, and verify how session regeneration after login prevents the attack.

Step 1 — Create the Project Folder

```
smailvm@smailvm:~$ mkdir session-lab1
cd session-lab1
mkdir app
```

A new project folder named session-lab1 was created. Inside it, an app folder was created to store the Node.js application files.

Step 2 — Create docker-compose.yml

```
 GNU nano 7.2                                    docker-compose.yml *
version: '3'

services:
  web:
    image: node:18
    container_name: session-lab1-app
    working_dir: /app
    volumes:
      - ./app:/app
    command: sh -c "npm install && node server.js"
    ports:
      - "8080:8080"
```

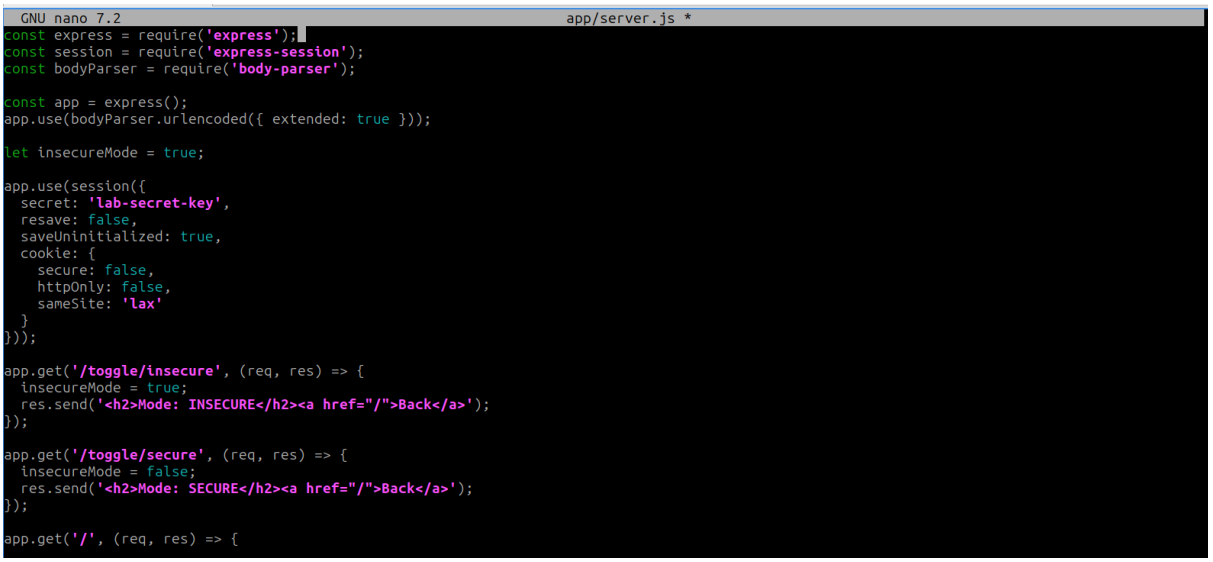
A Docker Compose file was created to run the lab application inside a Node.js 18 container. The local app folder was mounted inside the container, and port 8080 was exposed so the web application could be accessed from the browser.

Step 3 — Create app/package.json

```
 GNU nano 7.2                                    app/package.json *
{
  "name": "session-lab1",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.2",
    "express-session": "^1.17.3",
    "body-parser": "^1.20.2"
  }
}
```

The package.json file was created to define the Node.js project dependencies. The application uses Express for the web server, express-session for session handling, and body-parser to process form data.

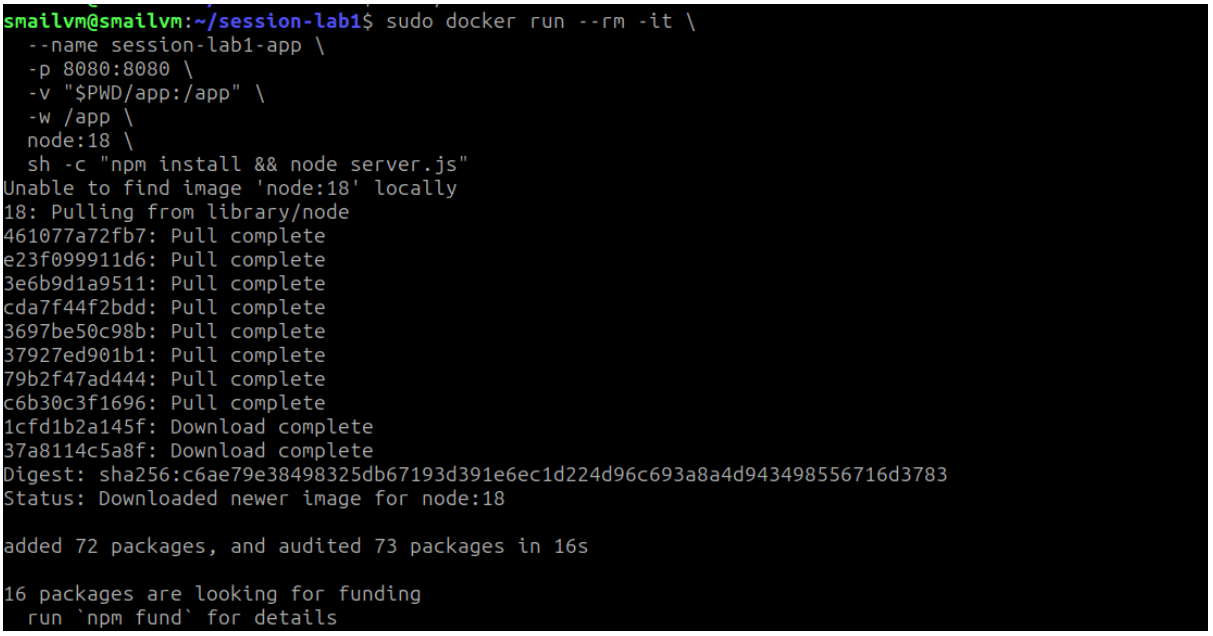
Step 4 — Create app/server.js



```
  GNU nano 7.2                                    app/server.js *
const express = require('express');
const session = require('express-session');
const bodyParser = require('body-parser');

const app = express();
app.use(bodyParser.urlencoded({ extended: true }));

let insecureMode = true;

app.use(session({
  secret: 'lab-secret-key',
  resave: false,
  saveUninitialized: true,
  cookie: {
    secure: false,
    httpOnly: false,
    sameSite: 'lax'
  }
}));

app.get('/toggle/insecure', (req, res) => {
  insecureMode = true;
  res.send('<h2>Mode: INSECURE</h2><a href="/">Back</a>');
});

app.get('/toggle/secure', (req, res) => {
  insecureMode = false;
  res.send('<h2>Mode: SECURE</h2><a href="/">Back</a>');
});

app.get('/', (req, res) => {
```

The server.js file defines a vulnerable session-based web application. In insecure mode, the application keeps the same session ID after login. In secure mode, it calls req.session.regenerate() after login, replacing the old session ID with a new one.

Step 5 — Launch the Lab



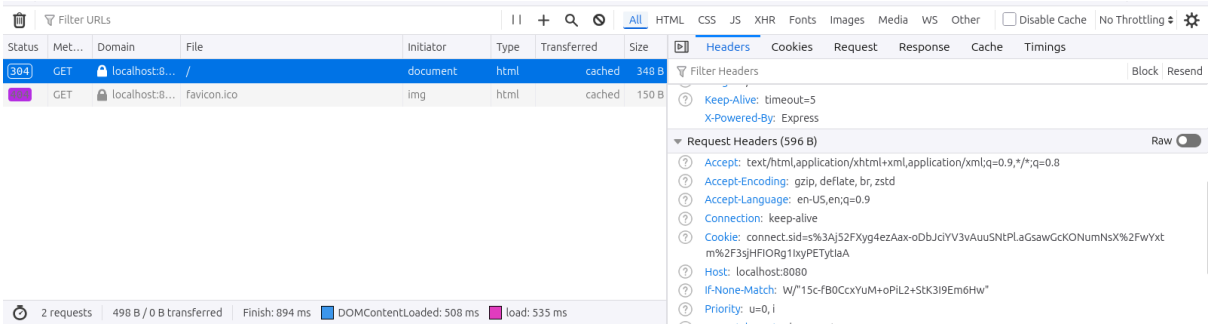
```
smailvm@smailvm:~/session-lab1$ sudo docker run --rm -it \
  --name session-lab1-app \
  -p 8080:8080 \
  -v "$PWD/app:/app" \
  -w /app \
  node:18 \
  sh -c "npm install && node server.js"
Unable to find image 'node:18' locally
18: Pulling from library/node
461077a72fb7: Pull complete
e23f099911d6: Pull complete
3e6b9d1a9511: Pull complete
cda7f44f2bdd: Pull complete
3697be50c98b: Pull complete
37927ed901b1: Pull complete
79b2f47ad444: Pull complete
c6b30c3f1696: Pull complete
1cfd1b2a145f: Download complete
37a8114c5a8f: Download complete
Digest: sha256:c6ae79e38498325db67193d391e6ec1d224d96c693a8a4d943498556716d3783
Status: Downloaded newer image for node:18

added 72 packages, and audited 73 packages in 16s

16 packages are looking for funding
  run `npm fund` for details
```

The Node.js session lab application was launched manually using docker run because Docker Compose was not working on the system. The container downloaded the node:18 image, installed the required npm packages, and successfully started the server on port 8080.

Step 6 — Verify the Web App

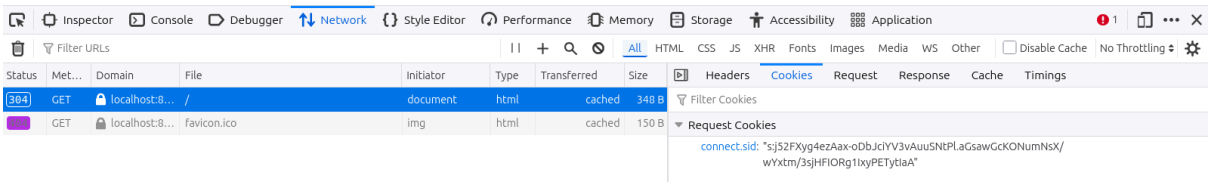
The application was accessed from the browser using http://localhost:8080. The home page loaded successfully and showed that the application was running in INSECURE mode.

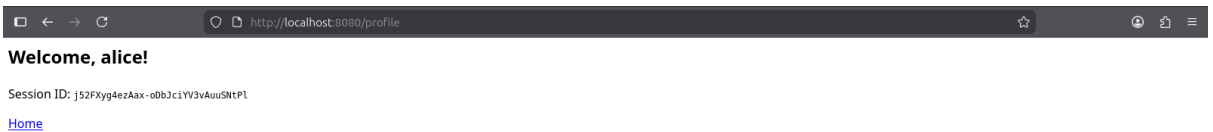
Step 7 — Observe Session Creation

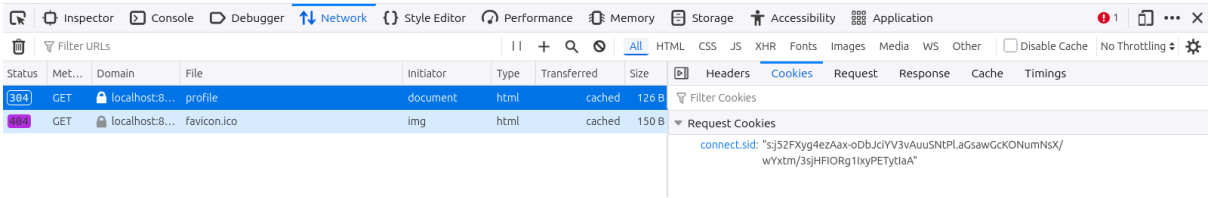


The connect.sid cookie was observed in the request headers. This confirms that the browser stored the session identifier and automatically sent it back to the server with later requests. Although the Set-Cookie header was not visible in this response, the presence of connect.sid in the request proves that a session cookie had already been created before login.

Step 8 — Login Behaviour and Session ID Persistence

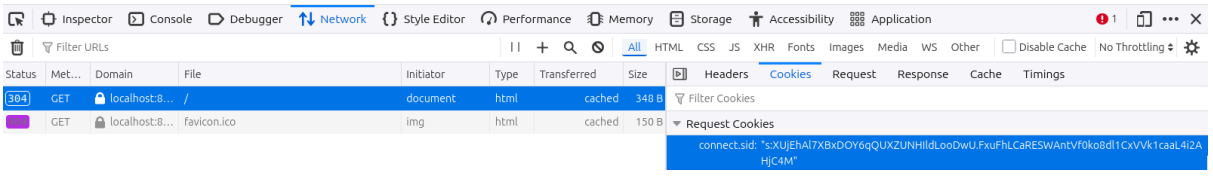




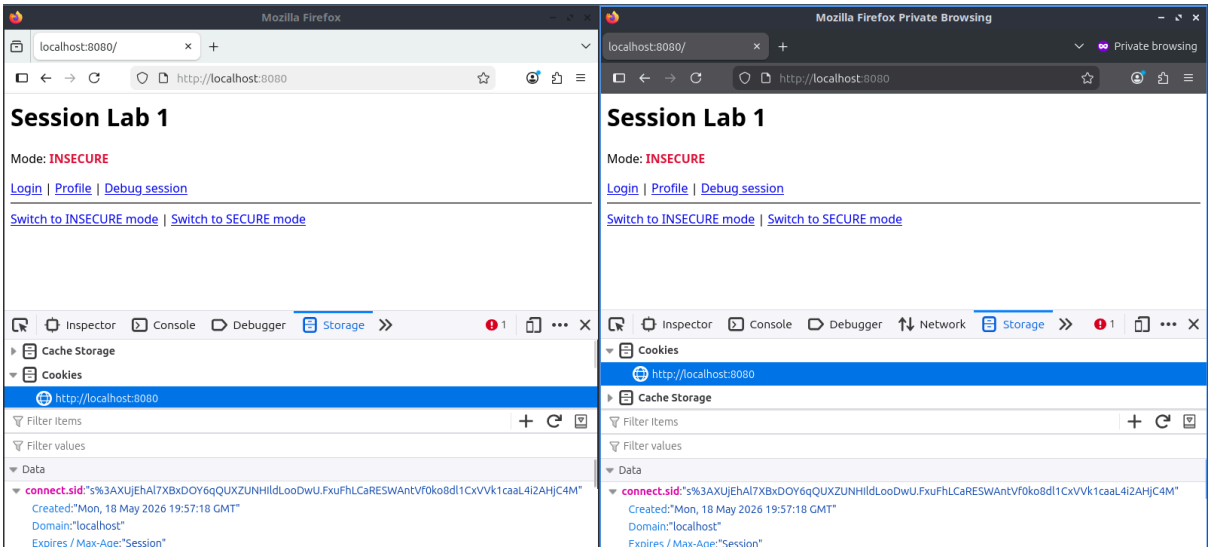


In insecure mode, the session ID did not change after login. The same connect.sid value existed before and after authentication. This behavior is dangerous because if an attacker knows the session ID before the victim logs in, the attacker can reuse that same session ID after the victim authenticates.

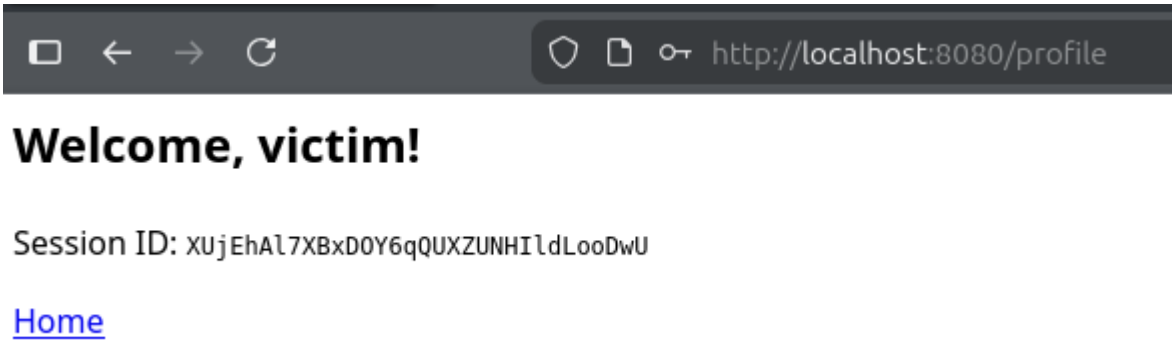
Step 9 — Session Fixation Attack



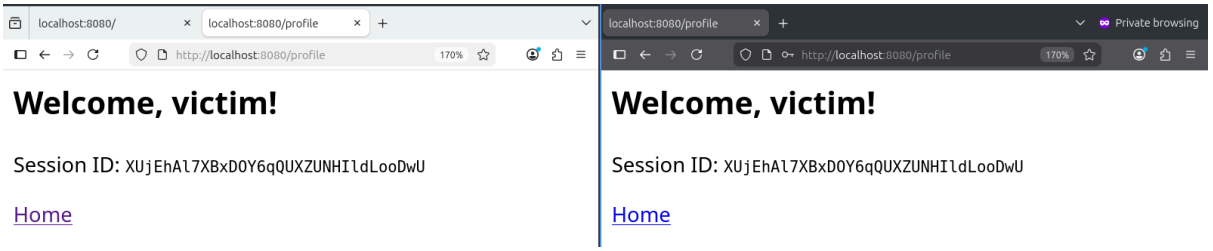
In Browser A, a pre-authentication session ID was observed. This session ID represents the value known by the attacker before the victim logs in.



Browser B now has the same connect.sid as Browser A



The victim logged in using Browser B after being forced to use the same session ID as Browser A. Since the application was in insecure mode, it did not regenerate the session ID after authentication.



A session fixation attack was performed in insecure mode. Browser A represented the attacker and obtained a pre-authentication connect.sid value. Browser B represented the victim. The victim browser was forced to use the same session ID, then logged in as victim. Because the application did not regenerate the session ID after login, Browser A was able to open /profile and access the victim's authenticated session.

Q1. What is the exact vulnerability that made this attack possible?

The vulnerability is that the application keeps the same session ID after login. It does not regenerate the session when the authentication state changes.

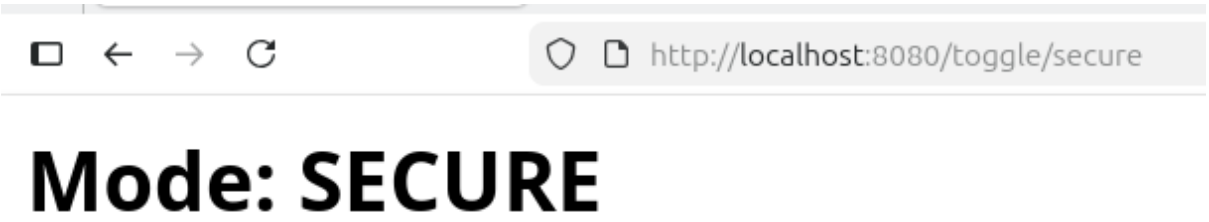
Q2. Did the attacker need to guess the session ID randomly?

No. The attacker did not need to guess it. The attacker already knew or controlled the session ID before the victim logged in.

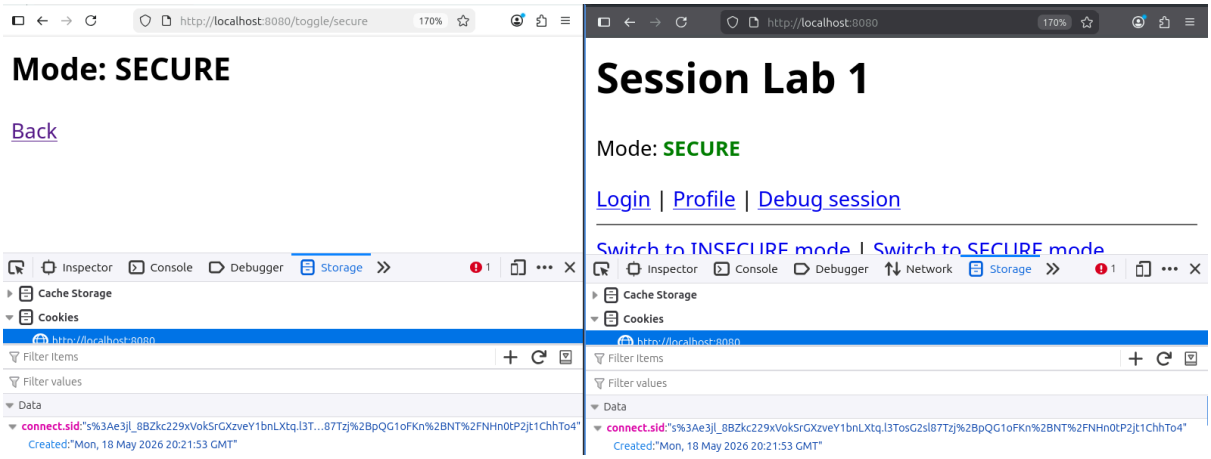
Q3. At which stage must the server apply the fix?

The server must regenerate the session immediately after successful authentication.

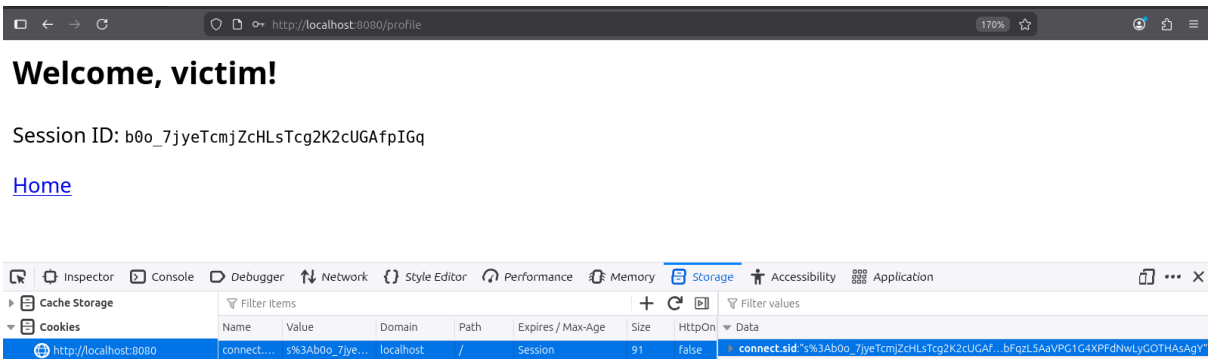
Step 10 — Session Regeneration as Defence



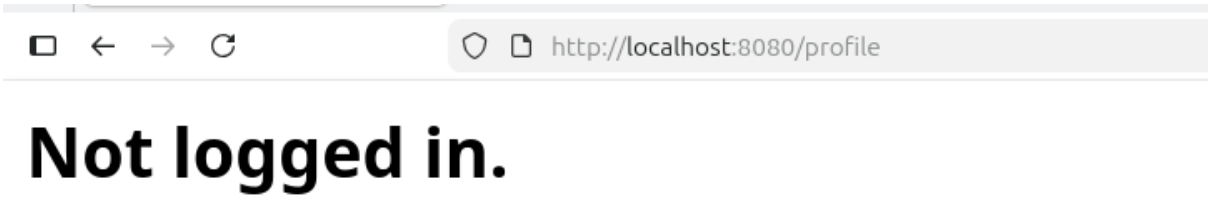
The application was switched to secure mode. In this mode, the server calls req.session.regenerate() after successful login, forcing the browser to receive a new session ID.



both browsers used the same pre-login session ID.



After the victim logged in while the application was in secure mode, the server regenerated the session. The victim received a new session ID, which was different from the attacker-known pre-login session ID.



The same fixation attack was repeated in secure mode. This time, after the victim logged in, the application regenerated the session ID. As a result, Browser B received a new authenticated session ID, while Browser A kept the old pre-authentication session ID. When Browser A accessed /profile, it was not authenticated. This proves that session regeneration prevents session fixation.

Q1. Why did the fixation attack fail in secure mode?

The attack failed because the server generated a new session ID immediately after login. The attacker's old session ID was no longer connected to the victim's authenticated session.

Q2. Why is session regeneration mandatory after authentication?

Session regeneration is mandatory because login changes the security state of the session. If the same session ID is kept after login, any attacker who knows that ID can reuse it. Regenerating the session breaks this link and protects the authenticated user.

Step 11 — Mini Assessment Answers

Q1. Define session fixation in your own words.

Session fixation is an attack where the attacker knows or controls a session ID before the victim logs in. If the application keeps the same session ID after login, the attacker can reuse that ID to access the victim's authenticated session.

Q2. Explain the difference between a pre-authentication session and an authenticated session.

A pre-authentication session exists before the user logs in. It may store temporary data but is not linked to an authenticated identity. An authenticated session exists after login and is linked to a specific user account.

Q3. Why is it dangerous to keep the same session ID after login?

It is dangerous because the session ID acts like a bearer token. If an attacker knows the session ID before login and it stays the same after login, the attacker can reuse it to impersonate the victim.

Q4. What does req.session.regenerate() do, and why does it stop fixation?

req.session.regenerate() creates a new session ID and replaces the old one. It stops fixation because the attacker's known pre-login session ID becomes useless after the victim logs in.

Q5. Explain the full attack chain of the fixation attack.

First, the attacker obtains a valid pre-authentication session ID from the application. Then the victim is forced to use that same session ID before logging in. In insecure mode, the application does not regenerate the session ID after login. When the victim authenticates, the same session ID becomes linked to the victim's account. The attacker then reuses that session ID and accesses the victim's authenticated profile.

Step 12 — Summary Table

Attack / Concept	Explanation	Main Defence
Session creation	The application creates a session and sends a connect.sid cookie to the browser.	Avoid unnecessary pre-authentication sessions and use secure cookie settings.
Session ID persistence	The same session ID remains before and after login.	Regenerate the session ID after authentication.
Session fixation	The attacker reuses a session ID known before the victim logs in.	Use req.session.regenerate() immediately after successful login.
Secure mode	The application creates a new session ID after login.	Old attacker-known session IDs become invalid.

Conclusion

This lab demonstrated how web sessions are created and managed using cookies. In insecure mode, the session ID remained the same after login, which allowed a session fixation attack. The attacker could reuse the victim's session ID and access the authenticated profile.

In secure mode, the application regenerated the session ID after login using req.session.regenerate(). This prevented the attacker from reusing the old session ID. Overall, the lab showed that session IDs must be regenerated whenever the authentication state changes.